

Einführung in Flutter

Was ist Flutter?

Flutter ist ein Open-Source-Framework von *Google* zur Entwicklung von plattformunabhängigen Anwendungen für Mobilgeräte, Web und Desktop. Es ermöglicht die Entwicklung von Anwendungen für Android, iOS, Windows, Mac, Linux und das Web aus einer einzigen Codebasis.

Es besteht aus zwei Hauptkomponenten:

- Ein **UI Framework** zum entwickeln von plattformübergreifenden Benutzeroberflächen.
- Einer **Sammlung von Tools** zum Entwickeln, Testen und Verwalten von Anwendungen.

Flutter verwendet die Programmiersprache Dart

Flutter verwendet eine eigene Programmiersprache mit dem Namen **Dart**. Dart ist eine objektorientierte Programmiersprache, die von Google entwickelt wurde. Sie ist recht ähnlich zu Java und JavaScript und wird von Flutter verwendet, um Anwendungen zu entwickeln.

Bevor wir also mit der Entwicklung von Flutter-Anwendungen beginnen, sollten wir uns mit der Programmiersprache Dart vertraut machen.

Weshalb ist Dart die Programmiersprache von Flutter?

Dart wurde von Google entwickelt und ist die bevorzugte Programmiersprache für die Entwicklung von Flutter-Anwendungen. Es bietet mehrere Vorteile:

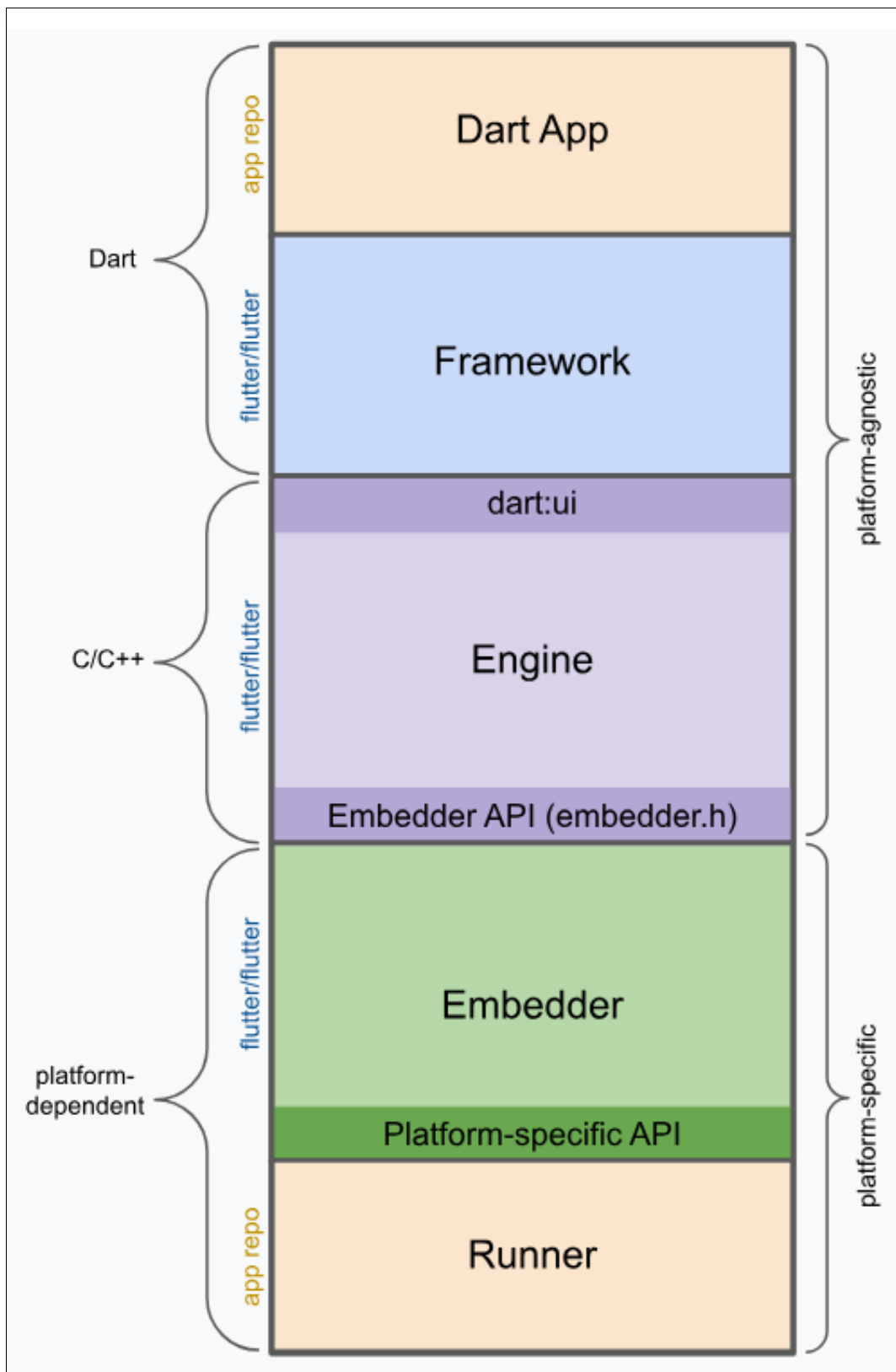
- **Leistung:** Dart ist eine kompilierte Sprache, die eine hohe Leistung bietet. Flutter-Anwendungen werden in nativen Code kompiliert, was zu einer schnellen Ausführung führt.

- **Hot Reload:** Dart unterstützt das Hot Reload-Feature von Flutter, das es Entwicklern ermöglicht, Änderungen am Code sofort in der laufenden Anwendung zu sehen, ohne die Anwendung neu starten zu müssen.
- **Einfachheit:** Dart ist eine einfach zu erlernende Sprache mit einer klaren Syntax, die es Entwicklern ermöglicht, schnell produktiv zu werden.
- **Plattformunabhängigkeit:** Dart ermöglicht die Entwicklung von plattformübergreifenden Anwendungen, die auf verschiedenen Betriebssystemen laufen können.
- **Große Community:** Dart hat eine wachsende Community von Entwicklern, die Ressourcen, Bibliotheken und Tools bereitstellen, um die Entwicklung von Anwendungen zu erleichtern.

Architektur von Flutter

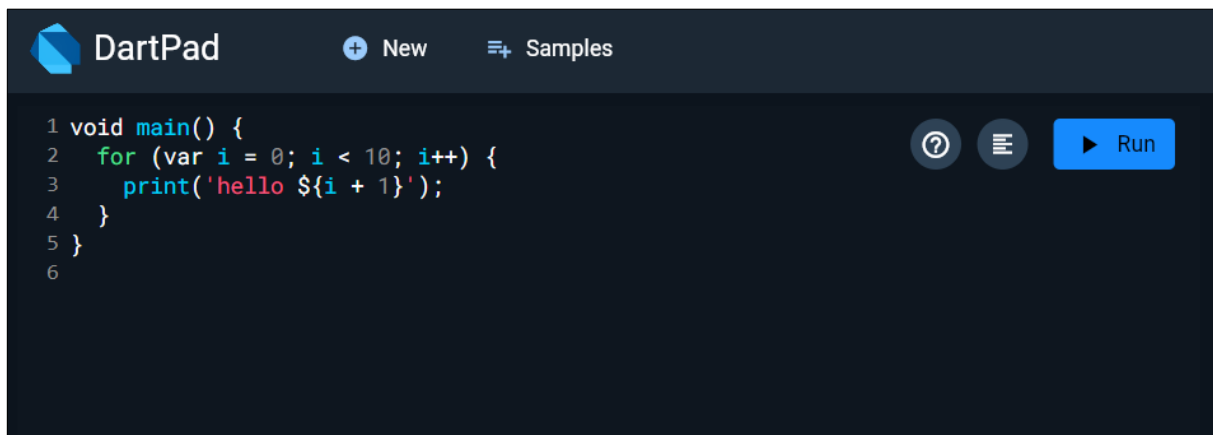
Flutter verwendet eine eigene Rendering-Engine namens **Skia**, die es ermöglicht, plattformunabhängige Benutzeroberflächen zu erstellen. Die Architektur von Flutter besteht aus mehreren Schichten:

- **Dart Framework:** Die oberste Schicht, die aus einer Sammlung von Bibliotheken und Widgets besteht, die zur Erstellung von Benutzeroberflächen verwendet werden.
- **Flutter Engine:** Die mittlere Schicht, die die Rendering-Engine Skia und andere wichtige Komponenten wie die Text-Rendering-Engine und die Grafikbibliothek enthält.
- **Plattform-Schnittstellen:** Die unterste Schicht, die die Kommunikation zwischen der Flutter-Anwendung und dem zugrunde liegenden Betriebssystem ermöglicht.



Einführung in Dart

Wir verwenden für die ersten Schritte in Dart den Online-Editor [DartPad](#). DartPad ist ein Online-Editor, der es ermöglicht, Dart-Code direkt im Browser auszuführen.



Im Beispielprogramm sieht man schon einige wichtige Elemente von Dart:

```
void main() {  
  for (var i = 0; i < 10; i++) {  
    print('hello ${i + 1}');  
  }  
}
```

- Methoden sehen aus wie in Java: `void main() { ... }`
- Variablen werden z.B. mit `var` deklariert: `var i = 0`
- *Strings* werden mit einfachen oder doppelten Anführungszeichen definiert: `'hello ${i + 1}'`. Mit `${}` können Variablen in Strings eingebettet werden.
- Die `for`-Schleife sieht aus wie in Java: `for (var i = 0; i < 10; i++) { ... }`
- `print` gibt Text auf der Konsole aus.

Variablen

Variablen in Dart können mit `var`, `final` oder `const` deklariert werden:

- `var` ist eine Variable, deren Typ zur Laufzeit festgelegt wird.
- `final` ist eine Variable, die nur einmal zugewiesen werden kann.
- `const` ist eine Konstante, die zur Kompilierzeit festgelegt wird.

```
var name = 'Max Mustermann';  
final now = DateTime.now(); // wird zur Laufzeit gesetzt  
const pi = 3.14159; // wird zur Kompilierzeit gesetzt
```

Datentypen

Dart ist eine statisch typisierte Programmiersprache. Sobald also eine Variable deklariert wurde, kann sie nicht mehr mit einem anderen Datentypen belegt werden. Es gibt folgenden Datentypen:

- `int` für Ganzzahlen
- `double` für Fließkommazahlen
- `String` für Zeichenketten
- `bool` für Wahrheitswerte
- `List` für Listen
- `Set` für Mengen
- `Map` für assoziative Arrays

```
int age = 42;  
double pi = 3.14159;  
String name = 'Max Mustermann';  
bool isStudent = true;  
List<int> numbers = [1, 2, 3, 4, 5];  
Map<String, int> scores = {'Alice': 100, 'Bob': 90, 'Charlie': 80};
```

Es gibt aber auch das Schlüsselwort `dynamic`, das es ermöglicht, Variablen mit unterschiedlichen Datentypen zu deklarieren:

```
dynamic value = 42;  
value = 'Hello, World!';
```

Dies sollte aber verhindert werden, da dadurch die Vorteile der statischen Typisierung verloren gehen (Compiler meldet uns frühzeitig, dass es potentiellen Fehlern kommen könnte).

Collections

Dart bietet verschiedene Arten von Sammlungen:

- `List` ist eine geordnete Sammlung von Elementen.
- `Set` ist eine ungeordnete Sammlung von eindeutigen Elementen.
- `Map` ist eine Sammlung von Schlüssel-Wert-Paaren.

```
var numbers = [1, 2, 3, 4, 5];  
var uniqueNumbers = {1, 2, 3, 4, 5};  
var scores = {'Alice': 100, 'Bob': 90, 'Charlie': 80};
```

Die Operationen auf Sammlungen sind ähnlich wie in Java:

```
var numbers = [1, 2, 3, 4, 5];  
numbers.add(6);  
numbers.remove(3);  
numbers.forEach((number) => print(number));
```

Hinweis: Bei der `remove`-Methode wird das Element selbst und nicht der Index übergeben.

Man kann aber natürlich auch ein Element an einem bestimmten Index hinzufügen oder entfernen:

```
var numbers = [1, 2, 3, 4, 5];  
numbers.insert(2, 6);  
numbers.removeAt(3);
```

Wie auch in Java oder C# sollte schon bei der Deklaration der Sammlung der Datentyp angegeben werden:

```
List<int> numbers = [1, 2, 3, 4, 5];  
Set<String> uniqueNumbers = {'1', '2', '3', '4', '5'};  
Map<String, int> scores = {'Alice': 100, 'Bob': 90, 'Charlie': 80};
```

Man kann auch mit `for-in` über die Elemente einer Sammlung iterieren:

```
var numbers = [1, 2, 3, 4, 5];  
for (var number in numbers) {  
    print(number);  
}
```

Weitere Beispiele:

// spread operator

```
final list1 = [1, 2, 3];  
final list2 = [0, ...list1]; // [0, 1, 2, 3]  
  
final list3 = [0, 1, 2];  
final list4 = [list3, 3, 4]; // [[0, 1, 2], 3, 4]  
final list5 = [0, ...list3, 3, 4]; // [0, 1, 2, 3, 4]  
  
final set1 = {1, 2, 3};  
final set2 = {0, ...set1}; // {0, 1, 2, 3}  
  
final map1 = {'a': 1, 'b': 2};
```

```
final map2 = {'c': 3, ...map1}; // {'c': 3, 'a': 1, 'b': 2}

// collection if

final isTrue = true;
final list6 = [1, if (isTrue) 2]; // [1, 2]

final isFalse = false;
final list7 = [1, if (isFalse) 2]; // [1]

// collection for

final list8 = [for (var i = 0; i < 3; i++) i]; // [0, 1, 2]

final list9 = [for (var i in [1, 2, 3]) i]; // [1, 2, 3]

final list10 = [for (var i in [1, 2, 3]) if (i > 1) i]; // [2, 3]

final list11 = [for (var i in [1, 2, 3]) if (i > 1) i * 2]; // [4, 6]
```

Collections filtern etc.

```
void main() {
  var numbers = [1, 2, 3, 4, 5, 6];

  // Filtern
  var evenNumbers = numbers.where((number) =>
    ↪ number.isEven).toList();
  print(evenNumbers); // [2, 4, 6]
```



```
// Mappen
var squaredNumbers = numbers.map((number) => number *
  ↪ number).toList();
print(squaredNumbers); // [1, 4, 9, 16, 25, 36]

// Reduzieren
var sum = numbers.reduce((a, b) => a + b);
print(sum); // 21
}
```

Null-Sicherheit

Dart unterstützt seit Version 2.12 die **Null-Sicherheit**. Das bedeutet, dass Variablen nicht mehr null sein können, es sei denn, sie sind explizit als nullable deklariert.

```
String name = 'Alice'; // Nicht null
String? message = null; // Kann null sein
```

Wenn eine Variable null sein kann, muss man das mit einem ? kennzeichnen:

```
String? message = null;
print(message.length); // Fehler: message kann null sein
```

```
if (message != null) {
  print(message.length); // OK
}
```

```
print(message?.length); // OK
```

Weiters Beispiel:

```
int? a; // = null
a ??= 3; // Wenn a null ist, wird 3 zugewiesen.
```

```
print(a); // <-- Prints 3.
```

```
a ??= 5; // Wenn a null ist, wird 5 zugewiesen.  
print(a); // <-- Still prints 3.
```

```
print(1 ?? 3); // <-- Prints 1.  
print(null ?? 12); // <-- Prints 12.
```

Funktionen

Funktionen sind in Dart wie in Java definiert:

- `void` gibt an, dass die Funktion keinen Wert zurückgibt.
- `int`, `double`, `String`, `bool`, `List`, `Map` geben den Rückgabebetyp an.

```
void greet(String name) {  
  print('Hello, $name!');  
}
```

```
int add(int a, int b) {  
  return a + b;  
}
```

Man kann Funktionen auch als Lambda-Ausdrücke definieren:

```
void main() {  
  var numbers = [1, 2, 3, 4, 5];  
  numbers.forEach((number) => print(number));  
}
```

Diese Funktionen können auch als Variablen gespeichert werden:

```
void main() {
```

```
var greet = (String name) => print('Hello, $name!');  
greet('Alice');  
}
```

Arrow-Funktionen können auch mehrere Anweisungen enthalten:

```
void main() {  
  var greet = (String name) {  
    print('Hello, $name!');  
    print('How are you?');  
  };  
  greet('Alice');  
}
```

Sie können auch bei der Funktionsdeklaration verwendet werden:

```
String greet(String name) => 'Hello, $name!';
```

Bei der Deklaration von Funktionen können auch benannte Parameter verwendet werden:

```
void greet({String name, String message}) {  
  print('$message, $name!');  
}  
  
void main() {  
  greet(name: 'Alice', message: 'Hello');  
}
```

Dieses Feature ist super wichtig später in Flutter, da viele Widgets viele Parameter haben und es so einfacher ist, die Parameter zu setzen.

Weitere Beispiele:

```
int sumUpToFive(int a, [int? b, int? c, int? d, int? e]) {  
  int sum = a;
```

```
    if (b != null) sum += b;
    if (c != null) sum += c;
    if (d != null) sum += d;
    if (e != null) sum += e;
    return sum;
}

// ...

int total = sumUpToFive(1, 2);
int otherTotal = sumUpToFive(1, 2, 3, 4, 5);

void printName(String firstName, String lastName, {String?
  ↪ middleName}) {
    print('$firstName ${middleName ?? ''} $lastName');
}

void main() {
    printName('Dash', 'Dartisan');
    printName('John', 'Smith', middleName: 'Who');
    // Named arguments can be placed anywhere in the argument list
    printName('John', middleName: 'Who', 'Smith');
}
```

Klassen

Klassen in Dart sehen aus wie in Java, jedoch gibt es keine Sichtbarkeitsmodifikatoren wie `public`, `private` oder `protected`. Attribute und Methoden sind standardmäßig `public`. Beginnt ein Attribut oder eine Methode mit einem Unterstrich `_`, ist es `private`.

```
class Person {
```

```
String name;
int age;
int _height;

Person(this.name, this.age) : height = Random().nextInt(200);

void greet() {
  print('Hello, $name! Your height is $_height cm.');
```

```
}
}
```

```
var alice = Person('Alice', 42);
alice.greet();
alice.age = 43;
// alice._height = 160;
```

- Im Konstruktor `Person(this.name, this.age)` werden die Attribute `name` und `age` initialisiert.
- Eine neue Instanz der Klasse wird mit `Person('Alice', 42)` erstellt, also ohne das Schlüsselwort `new`.

Weitere Beispiele:

```
class Spacecraft {
  String name;
  DateTime? launchDate;

  // Read-only non-final property
  int? get launchYear => launchDate?.year;

  // Constructor, with syntactic sugar for assignment to members.
  Spacecraft(this.name, this.launchDate) {
```

```
// Initialization code goes here.
}

// Named constructor that forwards to the default one.
Spacecraft.unlaunched(String name) : this(name, null);

// Method.
void describe() {
  print('Spacecraft: $name');
  // Type promotion doesn't work on getters.
  var launchDate = this.launchDate;
  if (launchDate != null) {
    int years = DateTime.now().difference(launchDate).inDays ~/
    ↪ 365;
    print('Launched: $launchYear ($years years ago)');
  } else {
    print('Unlaunched');
  }
}

var voyager = Spacecraft('Voyager I', DateTime(1977, 9, 5));
voyager.describe(); // Launch date is known.

var voyager2 = Spacecraft.unlaunched('Voyager II');
voyager2.describe(); // Launch date is unknown.
```

Named Constructors

```
class Point {
  double x, y;
```

```
Point(this.x, this.y);

Point.origin()
  : x = 0,
    y = 0;
}
var p1 = Point(2, 3);
var p2 = Point.origin();

print('p1: (${p1.x}, ${p1.y})'); // p1: (2.0, 3.0)
print('p2: (${p2.x}, ${p2.y})'); // p2: (0.0, 0.0)
```

Factory Constructors

```
class Square extends Shape {}

class Circle extends Shape {}

class Shape {
  Shape();

  factory Shape.fromTypeName(String typeName) {
    if (typeName == 'square') return Square();
    if (typeName == 'circle') return Circle();

    throw ArgumentError('Unrecognized $typeName');
  }
}

var shape1 = Shape.fromTypeName('square');
```

```
var shape2 = Shape.fromTypeName('circle');
```

Redirecting Constructors

```
class Automobile {  
  String make;  
  String model;  
  int mpg;  
  
  // The main constructor for this class.  
  Automobile(this.make, this.model, this.mpg);  
  
  // Delegates to the main constructor.  
  Automobile.hybrid(String make, String model) : this(make, model,  
    ↪ 60);  
  
  // Delegates to a named constructor  
  Automobile.fancyHybrid() : this.hybrid('Futurecar', 'Mark 2');  
}  
  
var car1 = Automobile('Toyota', 'Prius', 50);  
var car2 = Automobile.hybrid('Honda', 'Insight');  
var car3 = Automobile.fancyHybrid();
```

Getters und Setters

```
class MyClass {  
  int _aProperty = 0;  
  
  int get aProperty => _aProperty;
```



```
set aProperty(int value) {  
  if (value >= 0) {  
    _aProperty = value;  
  }  
}  
  
var myObject = MyClass();  
myObject.aProperty = 42; // Ruft den Setter auf  
print(myObject.aProperty); // Ruft den Getter auf
```

Vererbung

Vererbung in Dart funktioniert wie in Java:

```
class Student extends Person {  
  String university;  
  
  Student(String name, int age, this.university) : super(name, age);  
  
  void study() {  
    print('$name is studying at $university');  
  }  
}  
  
var bob = Student('Bob', 21, 'MIT');  
bob.greet();  
bob.study();
```

Die super-Methode ruft den Konstruktor der Basisklasse auf (ähnlich wie base in C#).

Weiter Beispiele:

```
class Orbiter extends Spacecraft {  
    double altitude;  
  
    Orbiter(super.name, DateTime super.launchDate, this.altitude);  
}  
var hubble = Orbiter('Hubble', DateTime(1990, 4, 24), 559);  
hubble.describe();
```

Hier wird der Konstruktor der Basisklasse Spacecraft mit `super.name` und `super.launchDate` aufgerufen (sieht man so häufiger als die herkömmliche Schreibweise mit `: super(...)`).

Interfaces

Interfaces in Dart werden mit dem Schlüsselwort `interface` definiert und mit `implements` implementiert:

```
abstract interface class Animal {  
    void eat();  
}  
  
class Dog implements Animal {  
    void eat() {  
        print('Dog is eating');  
    }  
}  
  
var dog = Dog();  
dog.eat();
```

In Dart, alle Klassen haben ein implizites Interface, das alle Methoden und Eigenschaften der Klasse enthält. Das bedeutet, dass alle Klassen als Interface verwendet werden können.

```
class Animal {  
  void eat() {  
    print('Animal is eating');  
  }  
}
```

```
class Dog implements Animal {  
  void eat() {  
    print('Dog is eating');  
  }  
}
```

```
void feed(Animal animal) {  
  animal.eat();  
}
```

```
var dog = Dog();  
feed(dog);
```

Interfaces können außerdem **nicht** erweitert werden, wie in Java.

Mixins

Mixins in Dart sind eine Möglichkeit, Code in eine Klasse einzufügen, ohne sie zu erweitern. Mixins sind eine Art von Mehrfachvererbung, die es einer Klasse ermöglicht, Methoden und Eigenschaften von mehreren Klassen zu verwenden.

```
mixin Fly {  
  void fly() {
```

```
        print('Flying');
    }
}

class Bird with Fly {
    void chirp() {
        print('Chirping');
    }
}

var bird = Bird();
bird.fly();
```

```
class Airplane with Fly {
    void takeOff() {
        print('Taking off');
    }
}

var airplane = Airplane();
airplane.fly();
```

Ein Mixin ist also eine Möglichkeit, Code-Teile (Funktionen und Eigenschaften) in mehreren Klassen wiederzuverwenden, ohne Vererbung zu erzwingen.

- Mixins ermöglichen es, Funktionen und Eigenschaften in Klassen zu "mischen", um die Funktionalität zu erweitern.
- Eine Klasse kann mehrere Mixins "mischen".
- Ein Mixin kann keine Konstruktoren haben.
- Mixins erben nicht automatisch von einer Oberklasse, es sei denn, sie verwenden die `on`-Klausel, um bestimmte Einschränkungen bezüglich der Basisklasse zu definieren.

```
mixin Flyable {  
  void fly() => print("I can fly!");  
}  
  
mixin Swimmable {  
  void swim() => print("I can swim!");  
}  
  
class Duck with Flyable, Swimmable {}  
  
void main() {  
  Duck duck = Duck();  
  duck.fly(); // Ausgabe: I can fly!  
  duck.swim(); // Ausgabe: I can swim!  
}
```

Weitere Beispiele:

```
mixin Piloted {  
  int astronauts = 1;  
  
  void describeCrew() {  
    print('Number of astronauts: $astronauts');  
  }  
}  
  
---  
  
class PilotedCraft extends Spacecraft with Piloted {  
  // ...  
}
```

Cascades

Dart unterstützt Cascades, die es ermöglichen, mehrere Operationen auf demselben Objekt durchzuführen, ohne das Objekt jedes Mal erneut zu referenzieren. Dies wird mit dem `..`-Operator erreicht.

```
var button = querySelector('#confirm');
button?.text = 'Confirm';
button?.classes.add('important');
button?.onClick.listen((e) => window.alert('Confirmed!'));
button?.scrollIntoView();
```

```
querySelector('#confirm')
  ?..text = 'Confirm'
  ..classes.add('important')
  ..onClick.listen((e) => window.alert('Confirmed!'))
  ..scrollIntoView();
```

async und await

Dart unterstützt asynchrone Programmierung mit den Schlüsselwörtern `async` und `await`. Dies ermöglicht es, langwierige Operationen auszuführen, ohne den Haupt-Thread zu blockieren.

```
const oneSecond = Duration(seconds: 1);
// ...
Future<void> printWithDelay(String message) async {
  await Future.delayed(oneSecond);
  print(message);
}
```

```
}
```

```
---
```

```
Future<void> createDescriptions(Iterable<String> objects) async {  
  for (final object in objects) {  
    try {  
      var file = File('$object.txt');  
      if (await file.exists()) {  
        var modified = await file.lastModified();  
        print(  
          'File for $object already exists. It was modified on  
          ↪ $modified.');        continue;  
      }  
      await file.create();  
      await file.writeAsString('Start describing $object in this  
        ↪ file.');    } on IOException catch (e) {  
      print('Cannot create description for $object: $e');    }  
  }  
}
```

```
---
```

```
Stream<String> report(Spacecraft craft, Iterable<String> objects)  
↪ async* {  
  for (final object in objects) {  
    await Future.delayed(oneSecond);
```

```
    yield '${craft.name} flies by $object';  
  }  
}
```

Exceptions

```
if (astronauts == 0) {  
  throw StateError('No astronauts.');
```



```
try {  
  breedMoreLlamas();  
} on OutOfLlamasException {  
  // A specific exception  
  buyMoreLlamas();  
} on Exception catch (e) {  
  // Anything else that is an exception  
  print('Unknown exception: $e');
```



```
} catch (e) {  
  // No specified type, handles all  
  print('Something really unknown: $e');
```



```
}
```

Dart-Referenzen

- [DartPad](#)
- [Dart Dokumentation](#)
- [Dart Cheat Sheet](#)

Übung

Mache dich weiter mit der Dart-Programmiersprache vertraut. Beantworte bzw. implementiere folgende Aufgaben. Verwende dazu auch DartPad, um die Beispiele auszuprobieren.

1. Null Safety und Typisierung

Implementiere eine Klasse `Profile`, bei der Felder wie `nickname` und `bio` nullable sein dürfen, aber z.B. die `email` zwingend notwendig ist.

Demonstriere, wie Dart mit Null-Sicherheit umgeht und wie sich dies von JavaScript und C# unterscheidet.

2. Konstruktoren: Named, Optional und Required Parameters

Erstelle einen Konstruktor mit named parameters für die Klasse `Profile`, wobei einige Parameter required und andere optional sein sollen (z.B. `Profile({required this.email, this.nickname, this.bio})`).

Zeige, wie die Initialisierung im Vergleich zu C# und JavaScript funktioniert.

3. Futures & Async/Await

Simuliere asynchrone Datenabfragen mit einer Funktion, die einen Future zurückgibt und mit `async/await` genutzt wird. Die Funktion soll z.B. nach einer Wartezeit einen neuen User zurückgeben.

Vergleiche das Handling von `async/await` in Dart mit `Task` in C# und `Promise` in JavaScript.

4. Mixins vs. Multiple Inheritance

Erstelle ein Mixin `Timestamped`, das eine Methode zum Loggen der aktuellen Zeit zur Verfügung stellt. Verwende es in einer Klasse `LoggableProfile` und erkläre den Unterschied zu Interfaces in C#, sowie zu Verhaltensmixins in JavaScript.

5. Factory-Konstruktor

Erstelle für die Klasse `Profile` einen factory constructor, der das gleiche Objekt aus unterschiedlichen Quellen (JSON, Map, etc.) erzeugen kann.

Diskutiere, wie Factory-Konstruktoren in Dart gegenüber Factory Patterns in C# und Konstruktorfunktionen in JavaScript funktionieren.